

TrackMaster Part-1

An Introductory Session for Beginners, Introducing them to the
Basics of Electronics and Microcontroller Programming

Managed by:

Robotics and Automation Society IEEE

Ibraheem Saboor	ChairPerson
Muhammad Zain	Co-Chair
Fatima Mubashar	Co-Chair

Supervised by: Sir Rehan Naeem



Department of Electrical Engineering
University of Engineering and Technology Lahore

December 2025

Contents

Abbreviations	iii
1 Overview	1
2 Introduction - Workshop1	2
3 Software Tutorial	4
3.1 Installing Required Software	4
3.2 Verify the Driver Installation	5
3.3 Blinking the On-Board LED	6
4 Experiment-1 : Interfacing GPIO Pins	8
4.1 Explanation	8
4.2 Objective	8
4.3 Components Required	8
4.4 Circuit Patching	8
4.5 Code	9
4.6 Oscilloscope Setup	10
4.7 Expected Output	10
4.8 Task-1	10
4.9 Task-2	10
5 Experiment-2 : Generation of PWM Signal	12
5.1 Explanation	12
5.2 Objective	13
5.3 Components	13
5.4 Circuit Patching	13
5.5 Code	13
5.6 Oscilloscope Setup	14
5.7 Expected Output	14
5.8 Task	15
6 Experiment-3 : Dynamic PWM Generation & Visualization	16
6.1 Explanation	16
6.2 Objective	16
6.3 Components	16
6.4 Circuit Patching	16
6.5 Code	17

6.6	Oscilloscope Setup	17
6.7	Expected Output	18
6.8	Task	18

Abbreviations

MCU	M icro C ontroller U nit
GPIO	G eneral P urpose I nput O utput
DIO	D igital I nput O utput
SRAM	S tatic R andom A ccess M emory
UART	U niversal A synchronous R eciever/ T ransmitter
SPI	S erial P eripheral I nterface
I2C	I nter I ntegrated C ircuit
ADC	A nalog to D igital C onverter
DAC	D igital to A nalog C onverter
PWM	P ulse W idth M odulation
VCC/VDD	P ositive S upply V oltage
GND/VSS	G round / R eference V oltage
IDE	I ntegrated D evelopment E nvironment
LED	L ight E mitting D iode

Chapter 1

Overview

This lab aims to provide anyone without any prior experience in electronics with a starting point to begin their journey. We will learn what a microcontroller is, how it works, what are its applications and what are its limitations. For this particular series of workshops, we will be using the Arduino Nano, an inexpensive entry level microcontroller development board equipped with all the functionality and tools that you might need for nearly every application at this level. This is the first part of a 3-part workshop, at the end of which, you will have made your own Line Following Robot. All the skills required to bring this project to completion will be taught during these 3 workshops. Each workshop will have its own set of objectives and goals:

- The first workshop will teach you basic MCU programming. You will learn to control outputs on GPIO pins and generate as well as control PWM signals.
- The second workshop will teach you input detection as well as active and passive polling. You will learn how to deal with noise and both level and edge triggered inputs.
- The third workshop will teach you how to assemble your own hardware and deal with all the problems that come with assembling physical circuits.

Chapter 2

Introduction - Workshop1

The MCU we will using is Atmega328p. The ATmega328 is an 8-bit AVR microcontroller running at 16 MHz. It features 32 KB of flash memory and 2 KB of SRAM. The chip includes 23 digital I/O pins, of which 6 support PWM and 8 can be used as ADC inputs with a 10-bit resolution. It also has periphery for SPI, I²C, UART. (Dont worry if you are not familiar with many of these terms. We will learn these one by one). Normally, these microcontrollers are used along with external components which work as either input or output devices. To make it easier for user to connect components with the MCU, the manufacturers provide development boards like the Arduino Uno and Arduino Nano. These boards have pin headers connect to all the useful pins of the MCU. They also have a serial communication IC which allows us to program the MCU using a simple USB cable which is also used to provide supply to the board. The pinout of the Arduino Nano is shown below: The DIO pins that are drawn with a wavy line can be used for

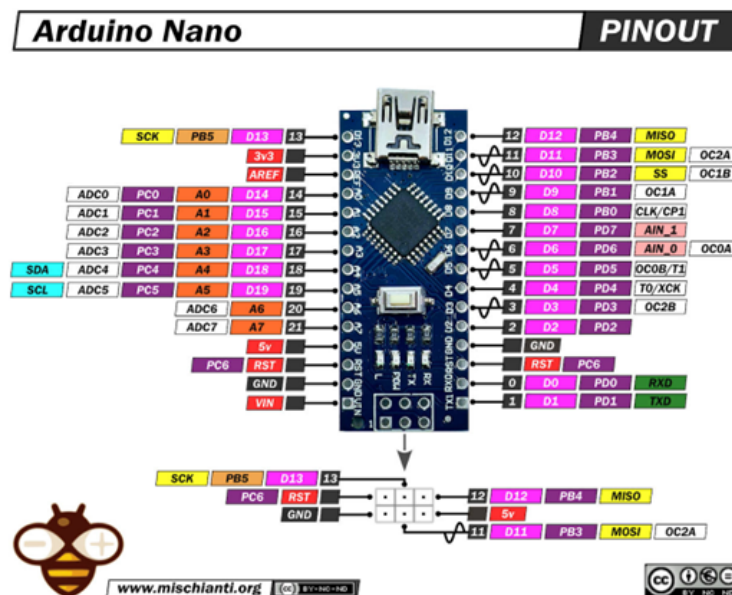


FIGURE 2.1: Arduino Nano Pinout

PWM functionality. The Arduino Nano will be connected to the computer using a Micro type-B USB cable. The development board also comes equipped with a built-in onboard led connected to D13. Since the on-board LED was already configured and pre-wired for us, it is easy to operate. The first task will be to download and setup the Arduino IDE software to program the Arduino Nano. The programming language that will be used will be C/C++. The main advantage of using arduino as opposed to other MCU options is the fact that arduino has a lot of libraries which makes the coding of many modules a lot simpler. This is will become clear as we start to interface different modules. Special care should be taken when working with MCU, the two main ways that MCU can be severely damaged are:

- **Overvoltage:** This can occur if we connect higher voltage than the controller can handle. E.g. we have a circuit which contains both 5V and 12V supply, the Arduino Nano is rated for around 5V – 6.5V. If we accidentally connect the 12V supply to our Arduino, it would instantly burn out the chip. (Arduino Nano is relatively cheap so it might not seem that big of an issue, but in a professional setting, you might be working with boards that cost over Rs 100,000 so it is a good practice to be cautious)
- **Overcurrent:** This occurs when we draw too much current from an output pin of the controller. Since our voltage is constant, the drawn current will be dictated by Ohms Law ($V = IR$) So current will depend on the resistance from the output pin to ground. The Arduino Nano has a max current rating of 40mA per I/O pin (above this would permanently damage the internal circuitry), the safe operating current for a digital I/O pin would be 20mA per pin. The max simultaneous current for all the digital I/O pins combined is 200mA (you can't draw 30mA from 10 pins simultaneously). A very common mistake that you might make is trying to patch the circuit while the supply is turned on, in doing so you might short circuit the controller ($R=0$ means $I=\text{infinity}$) causing overcurrent.

To avoid these mistakes, we will carefully design the circuit before powering it on.

Warning: Make sure the power is disconnected before modifying the circuit wiring and get your circuit approved from the available supervisor before powering on the circuit. In case of damage to any components, the student himself/herself will be responsible.

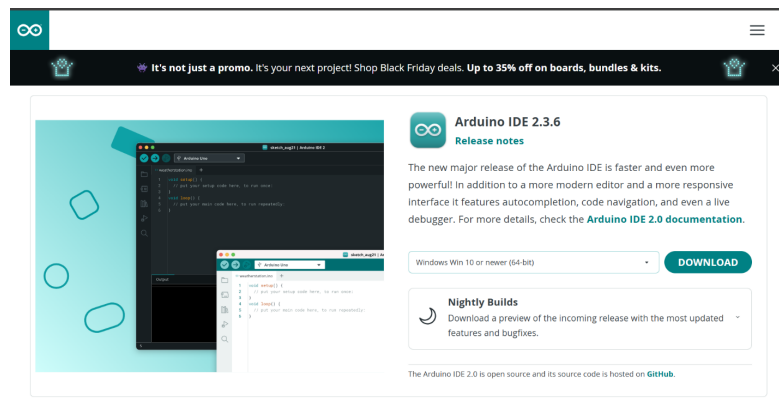
The solutions for the tasks in this manual can be found in this [drive](#).

Chapter 3

Software Tutorial

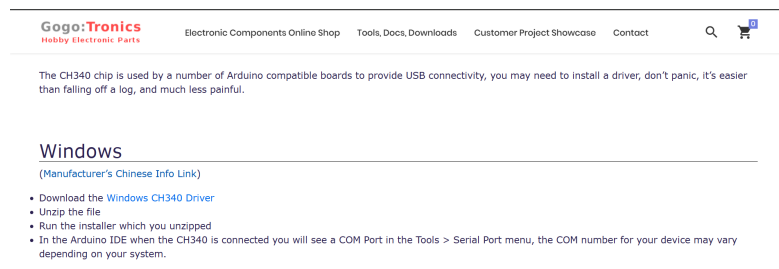
3.1 Installing Required Software

Download and install Arduino IDE using this [link](#). You can also follow this [tutorial](#) to download the IDE.



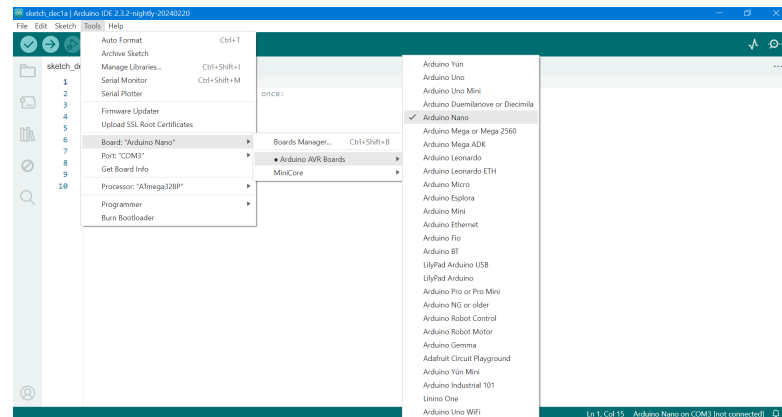
After downloading, double-click the installation file and continue with installation.

Download and install CH340 driver using this [link](#). And follow the instructions given in the image below.

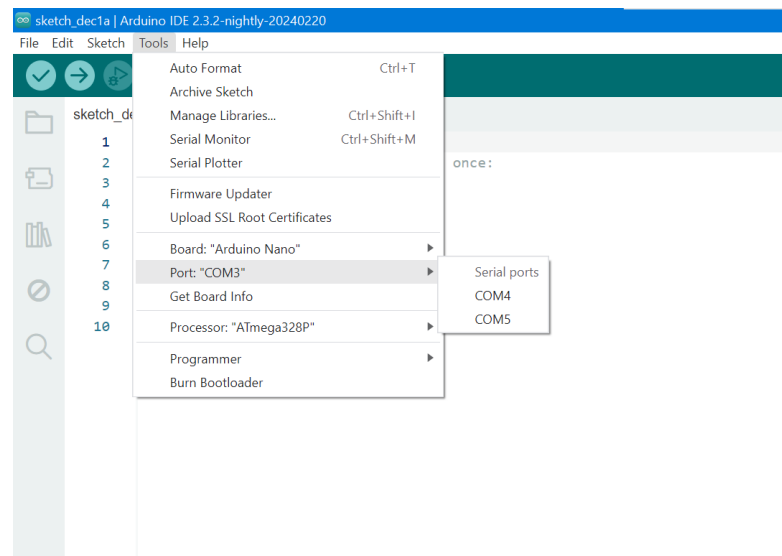


3.2 Verify the Driver Installation

First Open the Arduino IDE and connect Arduino Nano to your computer using the provided micro-USB cable. Now go to the tools tab and navigate to the required board manager as shown in the figure below:

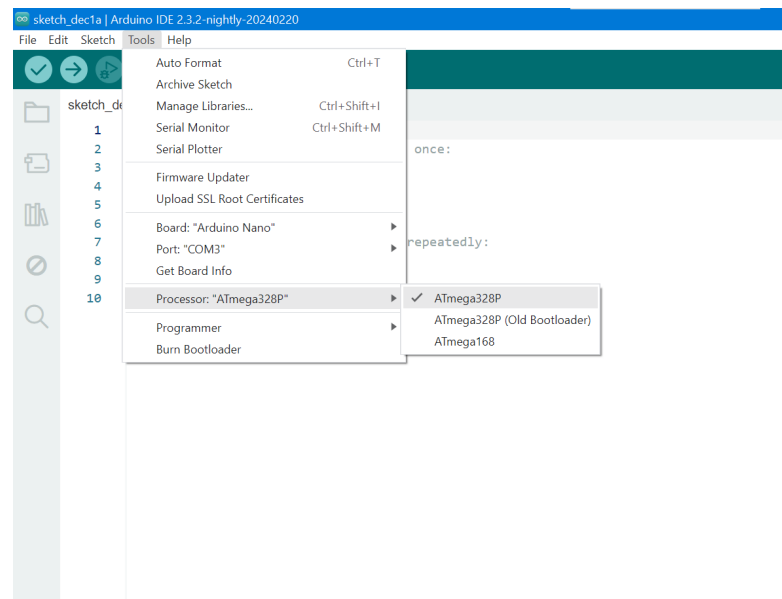


Make sure Arduino Nano is selected as in the Board tab. Now, choose the port your Nano is connected to in the menu as shown. If you don't know which port is being used, check the device manager on your computer.



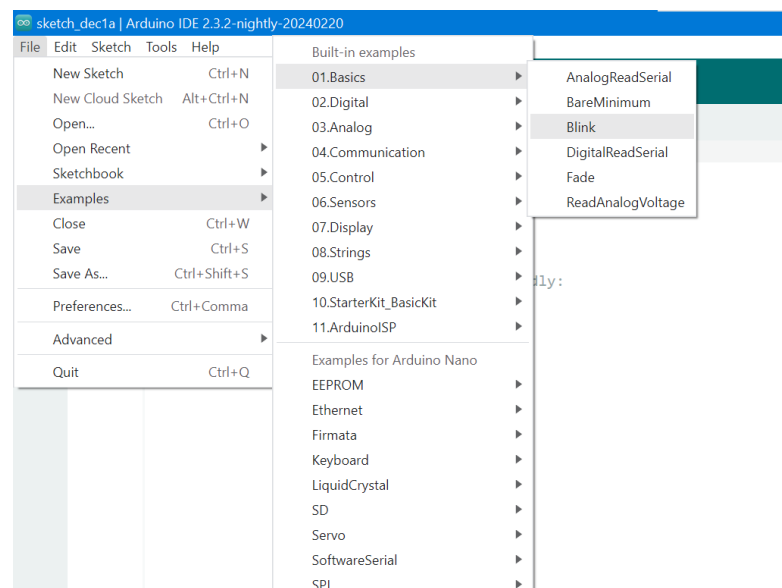
Choose Atmega328P in processor tab. If error occurs during the upload process, your Arduino Nano might be using old bootloader, select Atmega328P (Old Bootloader) in that case.

Now you will be able to see Arduino Nano connected a port in the bottom right corner of the window.

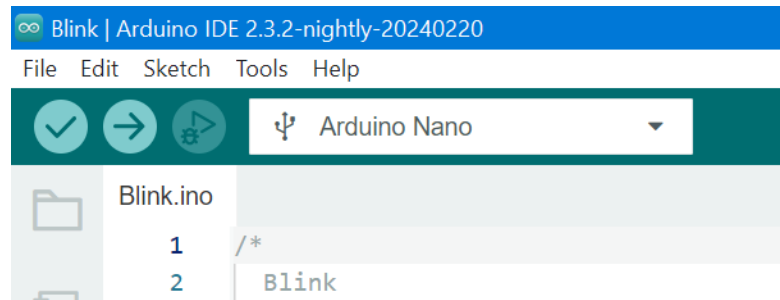


3.3 Blinking the On-Board LED

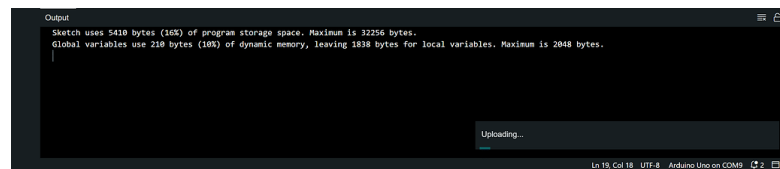
Arduino Nano has a built-in LED connected the digital pin 13. We will be using this LED to verify that we are on the right track. In the top left corner click on File and then Examples. Click on Basics and then finally Blink.



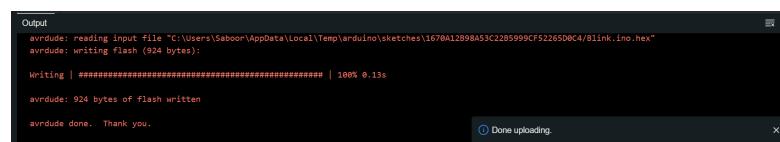
A new window will pop-up with a pre-written code. Now upload the code to the Arduino Nano. Click on the “Tick Icon” to verify the code. Verifying your code will check for any syntax errors and inform you if any are found. After verifying, Click the “Arrow Icon” to upload the code.



When the code is being uploaded, you will see the following text in the Output window:

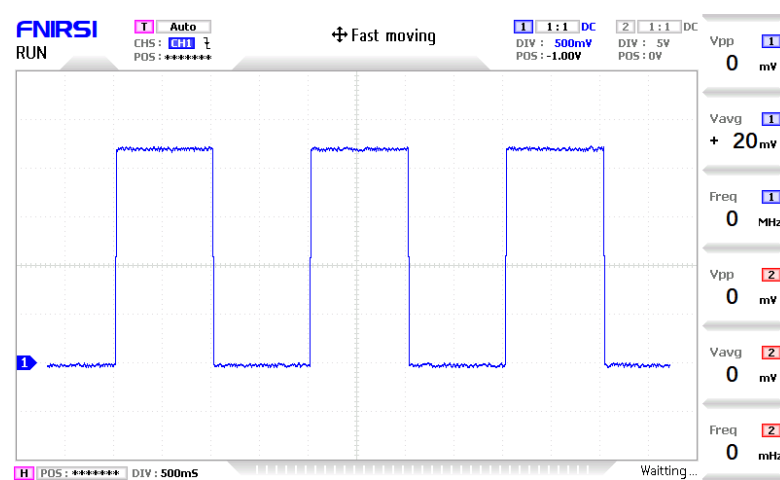


And if the code is successfully uploaded, you will see a popup like this at the bottom right of the screen:



The led on your Nano should be blinking now. The end result should look like the LED_BLINK.mp4 in the solutions drive.

The output on the oscilloscope should be same as:



Chapter 4

Experiment-1 : Interfacing GPIO Pins

4.1 Explanation

The digital pins of the controller have only two possible states (0 and 1). 0 states corresponds to when the pin is a 0V and 1 corresponds to when the pin is at VCC (VCC = 5V in our case). We can control the state of any DIO pin by programming our controller accordingly.

4.2 Objective

To blink an external LED by interfacing the DIO pins of the Arduino Nano.

4.3 Components Required

- Arduino Nano
- Laptop (To be arranged by individual)
- Breadboard (Available to be issued in the lab)
- LEDs (any colour)
- Resistors
- Jumper Wires
- Oscilloscope

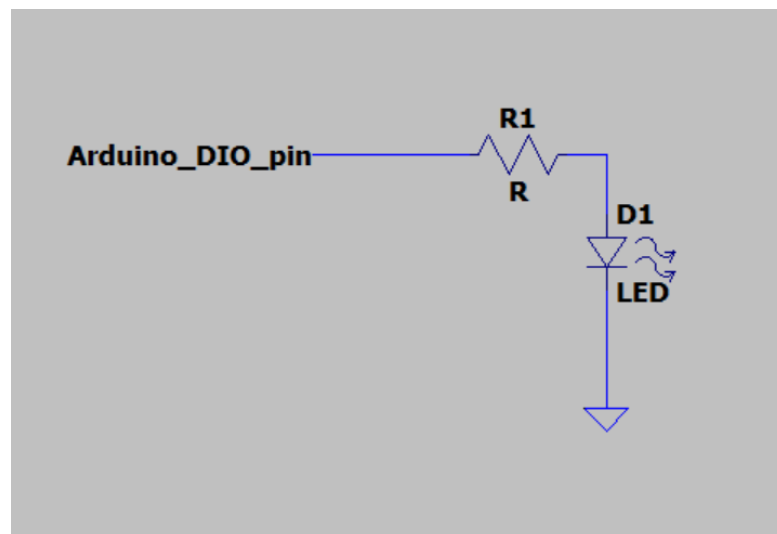
4.4 Circuit Patching

- Place the Arduino Nano in breadboard to easily access patch the circuit (make sure you are not shorting the pins).

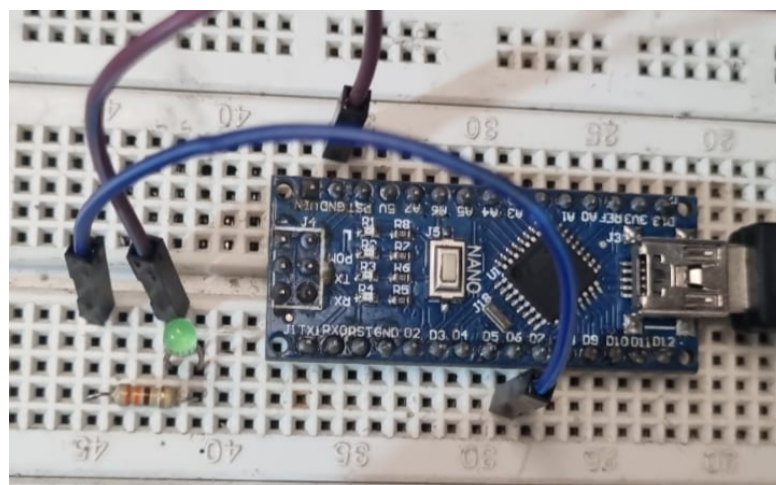
- Place an LED in series with a resistance between GND and one of the GPIO pins, ensuring cathode is connected to GND.

Here we encounter a design choice. What value of resistance do we use? We need to provide enough current to the led so that it glows visibly, but we also need to limit the current so that it doesn't cause overcurrent. The LED will need around 10mA-20mA to glow at normal brightness. Since this is within the safe operating limit of our controller, we can directly drive the LED from our digital I/O pin. Now use Ohms law to calculate the range for the value of the resistance to safely operate the LED and choose a value from that range that is available to you in the lab.

Now that you have the resistance, patch the circuit as follows:



The circuit on the breadboard should look similar to this:



4.5 Code

Open the Arduino IDE and write the code for LED to blink. A sample code is given below:

```
1 void setup() {  
2     pinMode(6, OUTPUT);  
3 }  
4  
5 void loop() {  
6     digitalWrite(6, HIGH);  
7     delay(1000);  
8     digitalWrite(6, LOW);  
9     delay(1000);  
10 }
```

4.6 Oscilloscope Setup

- Connect Probe Tip (Ch 1) to GPIO pin that you are using (Signal).
- Connect Ground Clip to Arduino GND (Reference).
- Set Vertical Scale to 2V or 5V / Div (Signal is 0V–5V).
- Determine suitable value of horizontal scale such that the waveform is clearly visible.

4.7 Expected Output

The end result should look like LED_EXTERNAL_BLINK.mp4 in the solutions drive.

4.8 Task-1

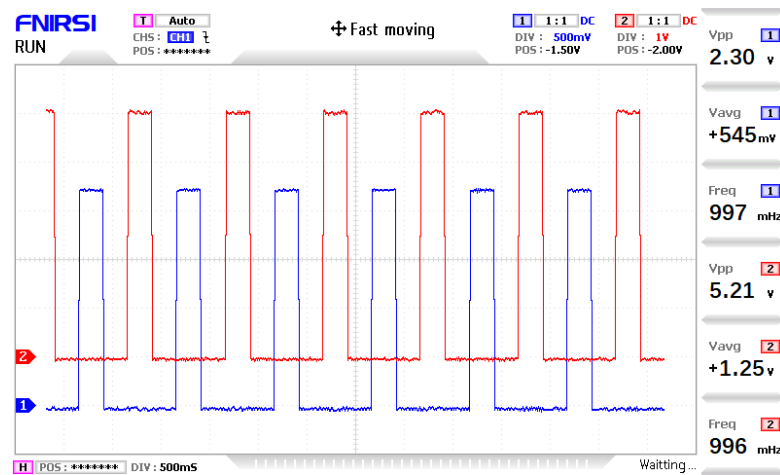
Adjust the code such that the led blinks faster. And show your lab supervisor. Now make it blink slower and show the supervisor. Observe both of these waveforms on the oscilloscope.

4.9 Task-2

Take two LEDs and make them blink in such a way that the first LED turns on for $\frac{1}{4}$ second while keeping the other off. Then, wait for $\frac{1}{4}$ second before turning the second LED on for $\frac{1}{4}$ second while the first remains off, followed by a final $\frac{1}{4}$ second wait. Observe the signals for both the LEDs simultaneously on the oscilloscope and show you supervisor.

The expected output can be observed in solutions drive in LED_ALTERNATE.mp4.

The output on the oscilloscope should be the same as shown in the image below



Chapter 5

Experiment-2 : Generation of PWM Signal

Until now, we have been controlling only whether the led is on or off. Next, we will learn how to control the brightness of the led. To achieve this, we will need to learn about analog signal and how to use PWM.

5.1 Explanation

As we discussed in the slides, there are 2 types of signals, digital and analog. The digital signals have only two values (5V and GND). Ideal analog signals have infinite possible values between 0 and VCC, but these signals are not possible to implement in reality. Modern analog signals typically have resolutions of 10 bits or 12 bits. A resolution of 10 bits means that there are (2^{10}) 1024 possible values for the analog output. Similarly, the 12-bit resolution will have 4096 possible values. These signals are usually generated by a DAC (digital to analog converter) module which takes in input digital values from your code and outputs analog signals. We might have an 8-bit DAC, 10-bit DAC, etc. The problem with DAC module is the cost. Although you might find multiple DAC drivers on more expensive controllers. The atmega328p being a budget friendly option does not have any DAC module to output analog voltages.

This is where PWM (pulse width modulation) is used. Instead of using a real analog signal, we use a pseudo-analog signal called a PWM signal. It is a digital signal with only 2 possible values, 0 and 1. It generates a pulse train at a high frequency, which when averaged out, gives us some value between 0 and 5V. And in most applications, it fulfils the role of analog voltages.

Not all the pins of the Arduino Nano are PWM compatible, you can refer to the pinout of the controller to identify the PWM pins. The PWM drivers on the Arduino Nano are 8-bit PWM drivers, so we have (2^8) 256 possible values of duty cycle.

5.2 Objective

Generate a PWM signal on a digital pin and visually observe various values of duty cycle (0%, 5%, 25%, 50%, 75%, 95%, 100%) on an LED and visually display the waveform on oscilloscope.

5.3 Components

- Arduino Nano
- Oscilloscope
- Resistor
- LEDs
- Breadboard
- Jumper Wires

5.4 Circuit Patching

- Place the Arduino Nano in breadboard to easily access patch the circuit (make sure you are not shorting the pins).
- Place an LED in series with a resistance between GND and one of the GPIO pins, ensuing cathode is connected to GND. This time make sure that the GPIO pin that you are using has PWM functionality. You can check using the pinout diagram given at the start of the manual.

5.5 Code

Open the Arduino IDE and write the code for LED to blink. A sample code is given below:

```
1  int pwmPin = 6;      // Define the LED connection to pin 6
2  int pwmValue = 0.25 * 255; // 8-bit PWM means max value of PWM is 255 so 25%
3                          //duty cycle will need PWM value of 0.25*255 (25% of 255).
4
5  void setup() {
6      pinMode(pwmPin, OUTPUT);    // Configure the pin to behave as an output
7  }
8
9  void loop() {
10     analogWrite(pwmPin, pwmValue); // Turn the LED on using PWM
11     delay(1000);                  // Pause for a second
12 }
13
```

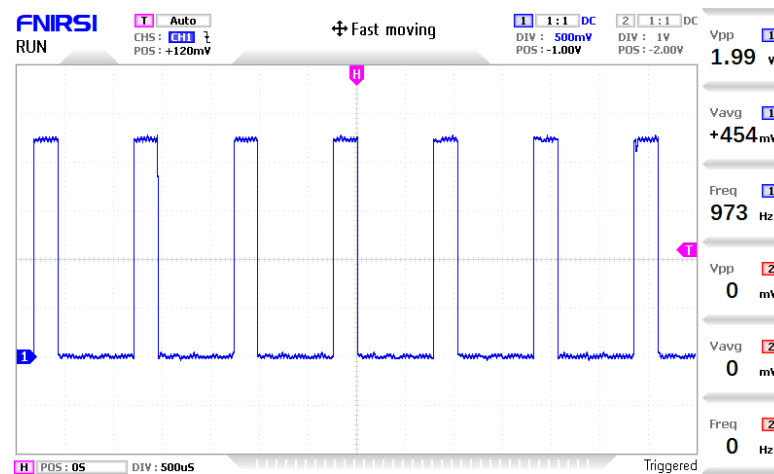
5.6 Oscilloscope Setup

- Connect Probe Tip (Ch 1) to pin 9 (Signal).
- Connect Ground Clip to Arduino GND (Reference).
- Set Vertical Scale to 2V or 5V / Div (Signal is 0V–5V).
- Determine suitable value of horizontal scale such that the waveform is clearly visible.

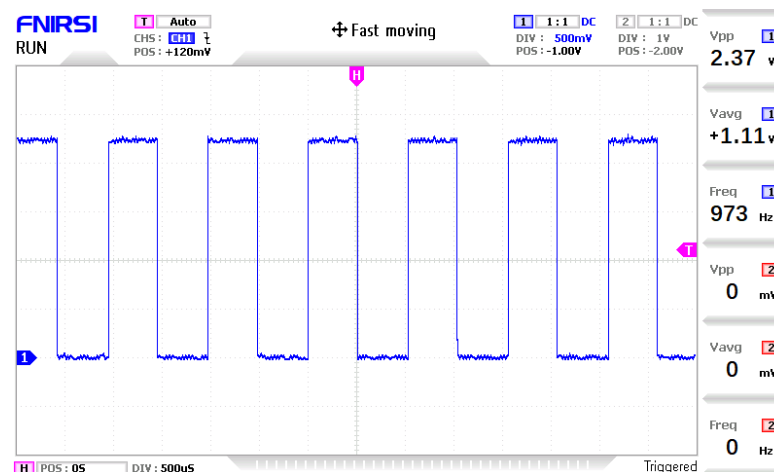
5.7 Expected Output

The outputs on the oscilloscope should be like these:

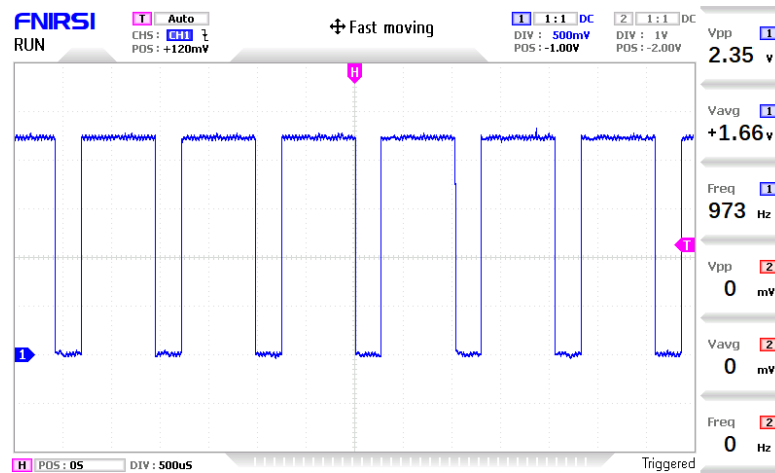
For 25% duty cycle:



For 50% duty cycle:



For 75% duty cycle:



5.8 Task

Answer the following questions:

- What changes do you observe on the LED when you change the duty cycle?
- What is the operating frequency of the PWM driver?
- What other applications can you think of for the PWM signals?

Chapter 6

Experiment-3 : Dynamic PWM Generation & Visualization

6.1 Explanation

Uptil now we have only generated a PWM signal of a fixed duty cycle.(i.e. we hardcoded a single value of duty cycle) and observed it on oscilloscope. Now we are going to dynamically change the dutycycle of the PWM signal such that we can observe different values of dutycycle without having to reprogram the Arduino Nano.

6.2 Objective

Programmatically vary the Duty Cycle of a PWM signal to create a fading effect on an LED and observe the changing pulse width on an oscilloscope.

6.3 Components

- Arduino Nano
- Oscilloscope
- Resistor
- LEDs
- Breadboard
- Jumper Wires

6.4 Circuit Patching

- Place the Arduino Nano in breadboard to easily access patch the circuit (make sure you are not shorting the pins).

- Place an LED in series with a resistance between GND and one of the GPIO pins, ensuing cathode is connected to GND. This time make sure that the GPIO pin that you are using has PWM functionality. You can check using the pinout diagram given at the start of the manual.

6.5 Code

Open the Arduino IDE and write the code for LED to blink. A sample code is given below:

```
1  int pwmPin = 9;    // LED connected to digital pin 9 (PWM pin)
2
3  void setup() {
4      pinMode(pwmPin, OUTPUT);
5  }
6
7  void loop() {
8
9      // FADE IN (0% to 100% Duty Cycle)
10     for (int fadeValue = 0; fadeValue <= 255; fadeValue += 5) {
11         analogWrite(pwmPin, fadeValue); // Update PWM width
12         delay(30);                      // Short pause to control fade speed
13     }
14
15     // FADE OUT (100% to 0% Duty Cycle)
16     for (int fadeValue = 255; fadeValue >= 0; fadeValue -= 5) {
17         analogWrite(pwmPin, fadeValue); // Update PWM width
18         delay(30);                      // Short pause to control fade speed
19     }
20
21 }
22
```

6.6 Oscilloscope Setup

- Connect Probe Tip (Ch 1) to pin 9 (Signal).
- Connect Ground Clip to Arduino GND (Reference).
- Set Vertical Scale to 2V or 5V / Div (Signal is 0V–5V).
- Determine suitable value of horizontal scale such that the waveform is clearly visible.

6.7 Expected Output

The output of the oscilloscope should be the same as BREATHING_LED_WAVE.mp4 in the solutions drive.

The led breathing effect can also be seen in BREATHING_LED.mp4 .

6.8 Task

- What would you change in the code to make the breathing effect of the LED either slower or faster?
- Modify the code so that whenever the LED goes to full brightness, it remains like that for a second and when it turns completely off, it stays off for a second. Observe this in the BREATHING_LED_HOLD.mp4 .
- Show your output and code changes to a supervisor.